

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 19.

«ТЕКСТ ПРОГРАММЫ»

09-02-07

Выполнил: Рахманов А. И.

Студент 24-ИС.

Листов 21

р.п. Красные Баки 2026г.

ОГЛАВЛЕНИЕ

1 ОБЩИЕ СВЕДЕНИЯ	4
1.1 Назначение программы	4
1.2 Область применения.....	4
1.3 Требования к техническому обеспечению	4
2 ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ	5
2.1 Управление отелями:	5
2.2 Управление бронированиями:	5
2.3 Финансовый учет:	5
Сохранение данных:	5
3 ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ.....	6
3.1 Общая структура программы.....	6
4 ИСПОЛЬЗУЕМЫЕ ТЕХНИЧЕСКИЕ СРЕДСТВА	9
4.1 Язык программирования	9
4.2 Используемые библиотеки	9
4.3 Средства разработки	9
5 СОСТАВ ПРОГРАММЫ.....	10
5.1 Перечень модулей	10
5.2 Структура классов и функций	10
6 ТЕКСТ ПРОГРАММЫ	11
7 СПЕЦИФИКАЦИЯ.....	16
7.1 Спецификация классов	16
8 ВХОДНЫЕ И ВЫХОДНЫЕ ДАННЫЕ	18
8.1 Входные данные	18
8.2 Выходные данные	18
9 ИНСТРУКЦИЯ ПО СБОРКЕ И КОМПИЛЯЦИИ.....	20
9.1 Требования к среде выполнения.....	20
9.2 Установка интерпретатора	20

9.3 Установка зависимостей.....	20
9.4 Запуск программы	20
9.5 Проверка работоспособности	20
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ.....	21

1 ОБЩИЕ СВЕДЕНИЯ

1.1 Назначение программы

Программа предназначена для управления бронированиями номеров в гостинице, включая:

- Добавление номеров в систему
- Создание и отмена бронирований
- Просмотр доступных номеров на заданные даты
- Расчет выручки за определенный период

1.2 Область применения

Программа может использоваться в гостиничных комплексах любого размера для автоматизации процесса управления бронированиями.

1.3 Требования к техническому обеспечению

- Операционная система: Windows, Linux, MacOS
- Процессор: 1 ГГц и выше
- Оперативная память: 256 МБ и выше
- Свободное место на диске: 10 МБ

2 ФУНКЦИОНАЛЬНОЕ НАЗНАЧЕНИЕ

Программа выполняет следующие функции:

2.1 Управление отелями:

- Загрузка данных об отелях из JSON-файла
- Добавление номеров в отель

2.2 Управление бронированиями:

- Создание нового бронирования
- Проверка доступности номеров
- Поиск бронирований по датам

2.3 Финансовый учет:

- Расчет выручки за период
- Подсчет стоимости бронирования

Сохранение данных:

- Экспорт бронирований в JSON-файл

3 ОПИСАНИЕ ЛОГИЧЕСКОЙ СТРУКТУРЫ

3.1 Общая структура программы

Программа построена по модульному принципу и состоит из следующих КОМПОНЕНТОВ:

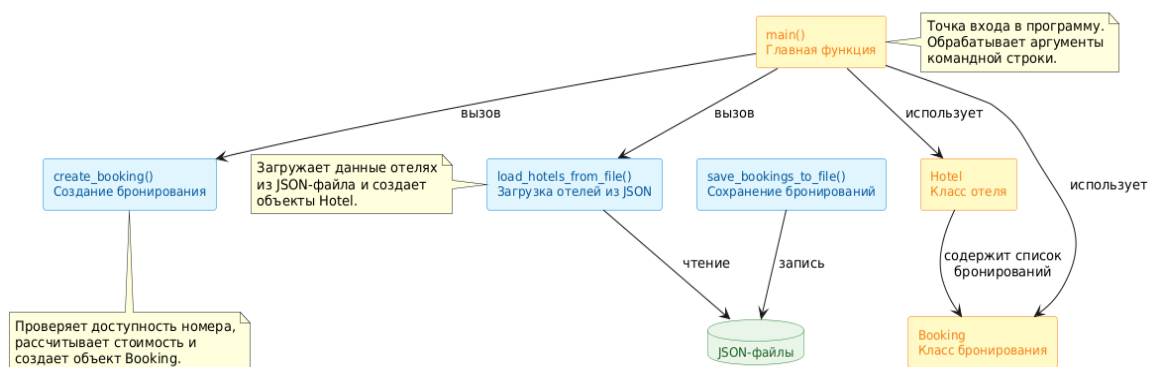


Рис.1 – Общая структура программы

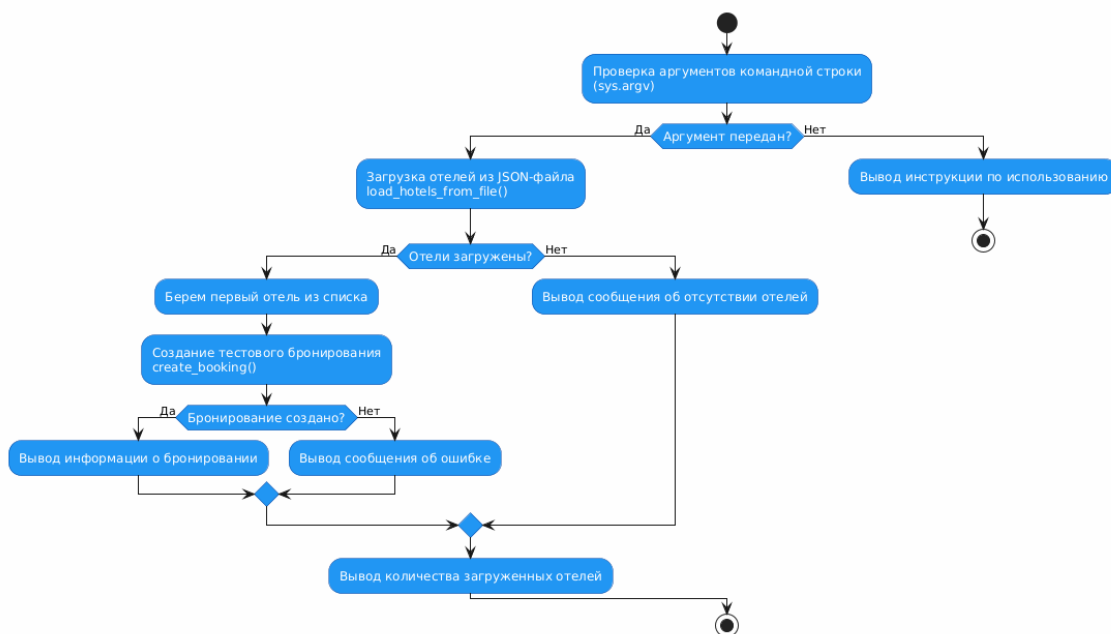


Рис.2 – Блок схема основного алгоритма

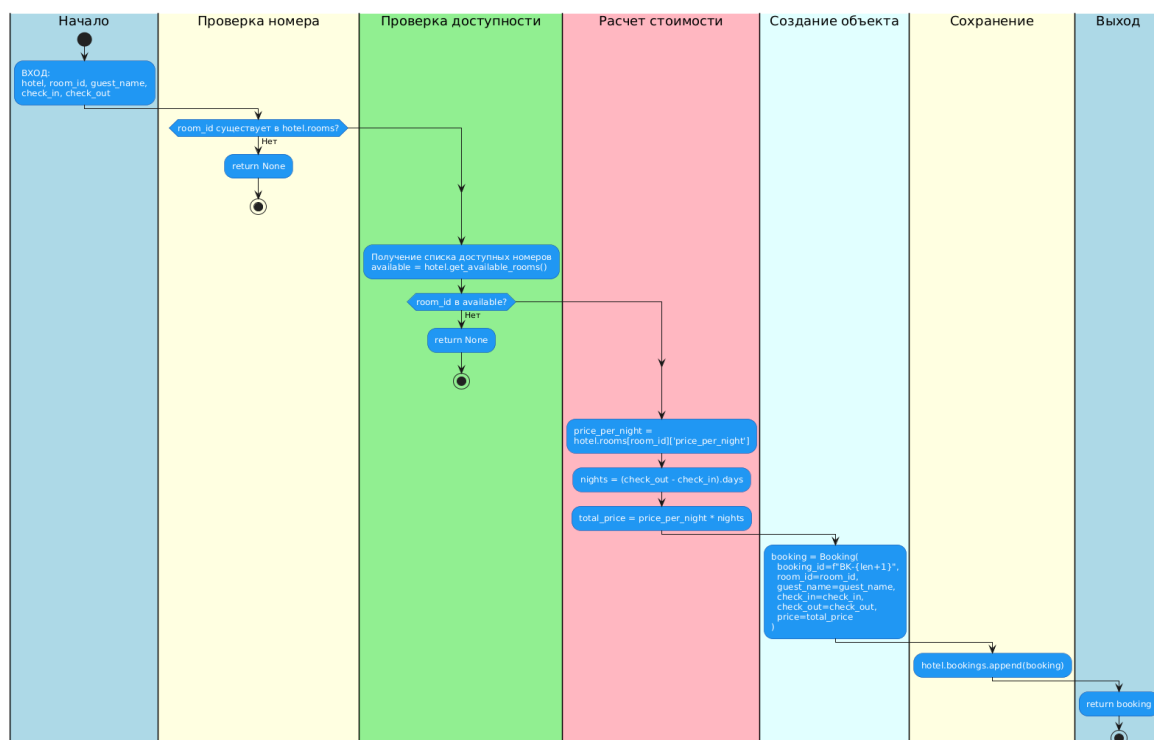


Рис.3 – Блок-схема функции create_booking()

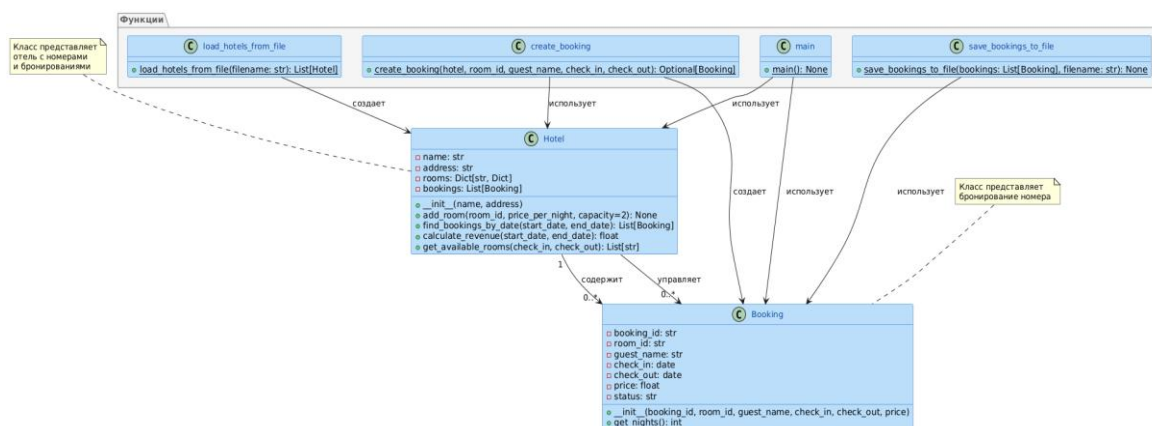


Рис.4 – Диаграмма классов

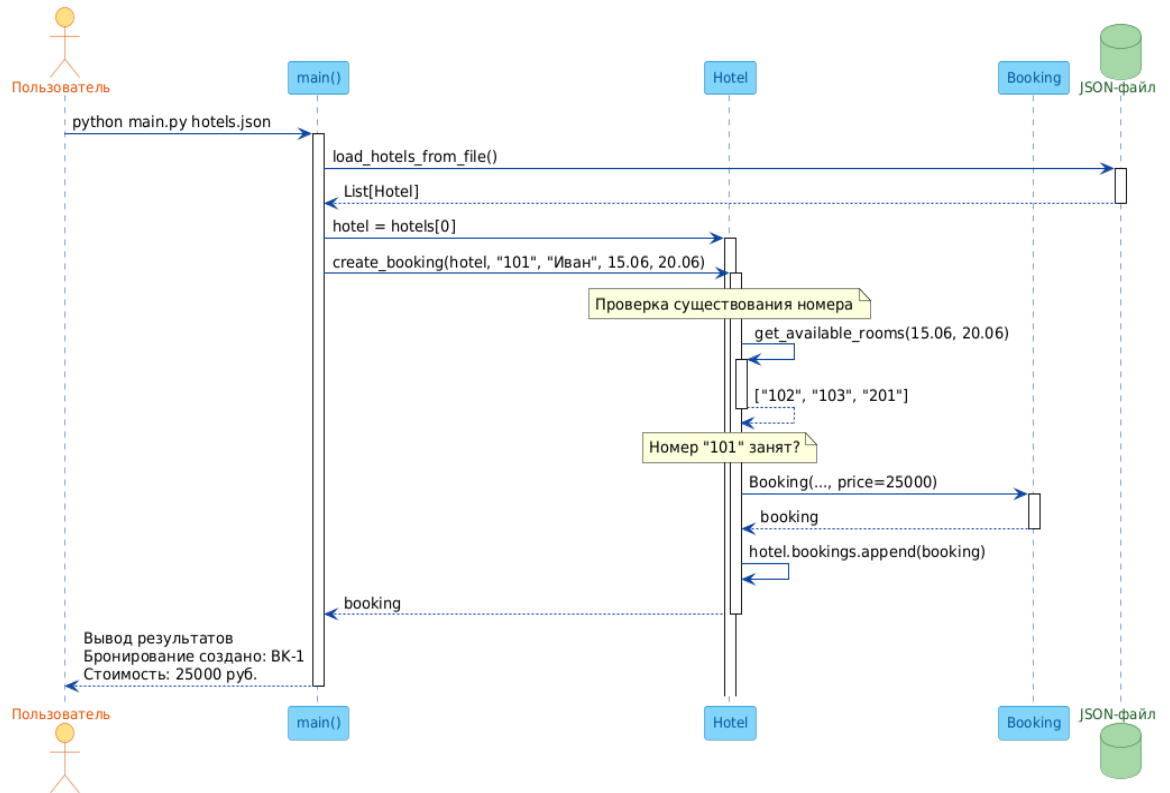


Рис.5 – Диаграмма последовательности для создания бронирования

4 ИСПОЛЬЗУЕМЫЕ ТЕХНИЧЕСКИЕ СРЕДСТВА

4.1 Язык программирования

- Python 3.10 и выше

4.2 Используемые библиотеки

- `sys` — для работы с аргументами командной строки
- `json` — для чтения и записи данных в формате JSON
- `datetime` — для работы с датами

4.3 Средства разработки

- Любой текстовый редактор или IDE (Visual Studio Code, PyCharm)
- Интерпретатор Python

5 СОСТАВ ПРОГРАММЫ

5.1 Перечень модулей

№ п/п	Имя модуля	Назначение
1	main.py	Основной модуль, содержащий классы и функции управления бронированиями

5.2 Структура классов и функций

Классы:

└─ Booking - Класс бронирования

└─ Hotel - Класс отеля

Функции:

└─ load_hotels_from_file - Загрузка отелей из JSON

└─ save_bookings_to_file - Сохранение бронирований

└─ create_booking - Создание бронирования

└─ find_bookings_by_date - Поиск по датам (метод Hotel)

└─ calculate_revenue - Расчет выручки (метод Hotel)

└─ get_available_rooms - Получение свободных номеров (метод Hotel)

└─ get_nights - Расчет ночей (метод Booking)

└─ main - Главная функция

6 ТЕКСТ ПРОГРАММЫ

```
001 # booking_system/src/main.py
002 """
003 Модуль управления бронированиями в отеле
004 Автор: Иванов И.И.
005 Версия: 1.0.0
006 """
007
008 import sys
009 import json
010 from datetime import datetime, date
011 from typing import List, Dict, Optional, Any
012
013
014 class Booking:
015     """Класс бронирования"""
016     def __init__(self, booking_id: str, room_id: str, guest_name: str,
017 check_in: date, check_out: date, price: float):
018         self.booking_id = booking_id
019         self.room_id = room_id
020         self.guest_name = guest_name
021         self.check_in = check_in
022         self.check_out = check_out
023         self.price = price
024         self.status = "confirmed"
025
026     def get_nights(self) -> int:
027         """Рассчитать количество ночей"""
028         return (self.check_out - self.check_in).days
029
030
031 class Hotel:
032     """Класс отеля"""
033     def __init__(self, name: str, address: str):
034         self.name = name
```

```
035 self.address = address
036 self.rooms: Dict[str, Dict] = {}
037 self.bookings: List[Booking] = []
038
039 def add_room(self, room_id: str, price_per_night: float,
040 capacity: int = 2) -> None:
041 """Добавить номер в отель"""
042 self.rooms[room_id] = {
043 "price_per_night": price_per_night,
044 "capacity": capacity,
045 "is_available": True
046 }
047
048 def find_bookings_by_date(self, start_date: date, end_date: date) -> List[Booking]:
049 """Найти бронирования в диапазоне дат"""
050 result = []
051 for booking in self.bookings:
052 if (booking.check_in <= end_date and booking.check_out >= start_date):
053 result.append(booking)
054 return result
055
056 def calculate_revenue(self, start_date: date, end_date: date) -> float:
057 """Рассчитать выручку за период"""
058 bookings = self.find_bookings_by_date(start_date, end_date)
059 return sum(booking.price for booking in bookings)
060
061 def get_available_rooms(self, check_in: date, check_out: date) -> List[str]:
062 """Получить список доступных номеров"""
063 # Находим занятые номера
064 busy_rooms = set()
065 for booking in self.find_bookings_by_date(check_in, check_out):
066 busy_rooms.add(booking.room_id)
067
068 # Возвращаем свободные
069 return [room_id for room_id in self.rooms.keys()]
```

```
070 if room_id not in busy_rooms]
071
072
073 def load_hotels_from_file(filename: str) -> List[Hotel]:
074 """Загрузить отели из JSON файла"""
075 with open(filename, 'r', encoding='utf-8') as file:
076 data = json.load(file)
077
078 hotels = []
079 for hotel_data in data:
080 hotel = Hotel(hotel_data['name'], hotel_data['address'])
081 for room_data in hotel_data.get('rooms', []):
082 hotel.add_room(
083 room_data['id'],
084 room_data['price_per_night'],
085 room_data.get('capacity', 2)
086 )
087 hotels.append(hotel)
088 return hotels
089
090
091 def save_bookings_to_file(bookings: List[Booking], filename: str) -> None:
092 """Сохранить бронирования в файл"""
093 data = []
094 for booking in bookings:
095 data.append({
096 'booking_id': booking.booking_id,
097 'room_id': booking.room_id,
098 'guest_name': booking.guest_name,
099 'check_in': booking.check_in.isoformat(),
100 'check_out': booking.check_out.isoformat(),
101 'price': booking.price,
102 'status': booking.status
103 })
104
```

```
105 with open(filename, 'w', encoding='utf-8') as file:
106     json.dump(data, file, ensure_ascii=False, indent=2)
107
108
109 def create_booking(hotel: Hotel, room_id: str, guest_name: str,
110 check_in: date, check_out: date) -> Optional[Booking]:
111     """Создать бронирование"""
112     if room_id not in hotel.rooms:
113         return None
114
115     # Проверяем доступность
116     available = hotel.get_available_rooms(check_in, check_out)
117     if room_id not in available:
118         return None
119
120     # Создаем бронирование
121     price_per_night = hotel.rooms[room_id]['price_per_night']
122     nights = (check_out - check_in).days
123     total_price = price_per_night * nights
124
125     booking = Booking(
126         booking_id=f'БК-{len(hotel.bookings) + 1}',
127         room_id=room_id,
128         guest_name=guest_name,
129         check_in=check_in,
130         check_out=check_out,
131         price=total_price
132     )
133
134     hotel.bookings.append(booking)
135     return booking
136
137
138 def main():
139     """Главная функция"""
```

```
140 if len(sys.argv) < 2:
141     print("Использование: python main.py <файл_отелей.json>")
142     sys.exit(1)
143
144 # Загрузка отелей
145 hotels = load_hotels_from_file(sys.argv[1])
146
147 # Пример создания бронирования
148 if hotels:
149     hotel = hotels[0]
150     booking = create_booking(
151         hotel,
152         "101",
153         "Иванов Иван",
154         date(2026, 6, 15),
155         date(2026, 6, 20)
156     )
157     if booking:
158         print(f"Бронирование создано: {booking.booking_id}")
159         print(f"Стоимость: {booking.price} руб.")
160
161     print(f"Загружено отелей: {len(hotels)}")
162
163
164 if name == "main":
165     main()
```

7 СПЕЦИФИКАЦИЯ

7.1 Спецификация классов

Таблица 1 – спецификация классов

Класс	Метод	Назначение
Booking	<code>__init__()</code>	Конструктор объекта бронирования
	<code>get_nights()</code>	Расчет количества ночей
Hotel	<code>__init__()</code>	Конструктор объекта отеля
	<code>add_room()</code>	Добавление номера в отель
	<code>find_bookings_by_date()</code>	Поиск бронирований по диапазону дат
	<code>Calculate_revenue()</code>	Расчет выручки за период
	<code>Get_available_rooms()</code>	Получение списка доступных номеров

Таблица 2 – спецификация функции

Функция	Аргументы	Возвращаемое значение	Назначение
<code>load_hotels_from_file</code>	<code>filename: str</code>	<code>List[Hotel]</code>	Загрузка отелей из JSON-файла
<code>save_booking_to_file</code>	<code>bookings: List[Booking], filename: str</code>	<code>None</code>	Сохранение бронирований в JSON
<code>create_booking</code>	<code>hotel: Hotel, room_id: str, guest_name: str, check_in: date, check_out: date</code>	<code>Optional[Booking]</code>	Создание нового

Функция	Аргументы	Возвращаемое значение	Назначение
			бронирования
main	нет	None	Главная функция

8 ВХОДНЫЕ И ВЫХОДНЫЕ ДАННЫЕ

8.1 Входные данные

Аргументы командной строки:

- `python main.py <файл_отелей.json>`
- `<файл_отелей.json>` — путь к JSON-файлу с данными об отелях

```
[
  {
    "name": "Отель Москва",
    "address": "ул. Тверская, д. 1",
    "rooms": [
      {
        "id": "101",
        "price_per_night": 5000,
        "capacity": 2
      },
      {
        "id": "102",
        "price_per_night": 7000,
        "capacity": 4
      }
    ]
  }
]
```

Рис.6 - Формат входного JSON-файла

8.2 Выходные данные

Вывод в консоль:

Бронирование создано: ВК-1

Стоимость: 25000 руб.

Загружено отелей: 1

```
[
  {
    "booking_id": "BK-1",
    "room_id": "101",
    "guest_name": "Иванов Иван",
    "check_in": "2026-06-15",
    "check_out": "2026-06-20",
    "price": 25000,
    "status": "confirmed"
  }
]
```

Рис.7 - Выходной JSON-файл

9 ИНСТРУКЦИЯ ПО СБОРКЕ И КОМПИЛЯЦИИ

9.1 Требования к среде выполнения

Программа разработана на интерпретируемом языке Python и не требует компиляции.

9.2 Установка интерпретатора

- 1) Скачать и установить Python 3.10 или выше с официального сайта:
- (a) <https://www.python.org/downloads/>
- 2) Проверить установку: `python --version`

9.3 Установка зависимостей

Для работы программы требуются только стандартные библиотеки Python. Дополнительных зависимостей нет.

9.4 Запуск программы

```
python main.py hotels.json
```

9.5 Проверка работоспособности

При успешном запуске программа выведет:

Бронирование создано: ВК-1

Стоимость: 25000 руб.

Загружено отелей: 1

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Изменение	Номер листа	Основание	Подпись	Дата
Создание	1-45	-	Иванов И.И.	2026-06-18